that attention heads (this is the same method implemented by Todd et al. (2024)). Like Todd et al. (2024), we do not apply any further scaling or normalization. For a given cluster c, we denote all tokens that are a part of this cluster by $T_j[c] \subseteq \{1, \ldots, T_j\}$. We then compute the average activity in each attention head over all those tokens:

$$\overline{Z}[c] := \frac{1}{\sum_{j=1}^{n} |T_j[c]|} \sum_{j \in T_j[c]} Z^{(j)} \in \mathbb{R}^{L \times H \times D}. \tag{6}$$

Finally, we compute the primitive vector $v^{(p)}[c]$ corresponding to this cluster as the average across the 35 attention heads with the largest AIE:

$$v^{(p)}[c] = \frac{1}{|\mathcal{A}|} \sum_{l,h \in \mathcal{A}} \overline{Z}_{lh} \in \mathbb{R}^{D}. \tag{7}$$

B.7 Time complexity

Our time complexity is determined by two main processes:

1. **CoT Generation + Activation Collection:** $O(L^2)$, where L is the sequence length (number of tokens in the reasoning trace). Complexity is quadratic because Transformer self-attention computes all token-to-token interactions. This happens once per response to generate the trace and extract internal activations. Typical responses contain between 500 and 6000 tokens; at that scale, the activation collection is the most expensive step for our method, taking between 5–10 minutes on 4 A40 GPUs (we note that multiple GPUs are required to analyze long responses).
2. **Clustering:** $O(K \times N \times I)$, with K the number of clusters ($K = 50$ usually), N the number of datapoints (activations collected), and I the number of iterations until convergence. Scaling is linear with each factor. This is much faster than activation collection: we measured 61 seconds for 10 responses with 50 clusters (linear, $\tilde{1}$ min with a CPU).

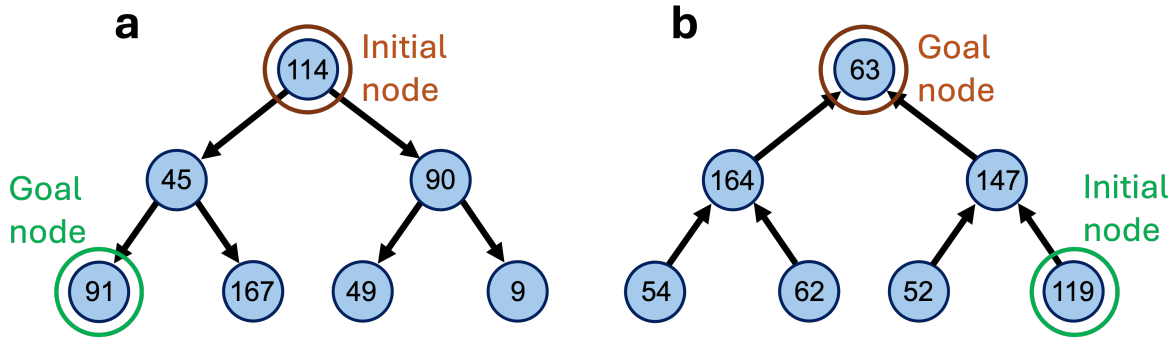

Figure 6: Examples of binary trees used in the two conditions. **a** Forward condition: models were tasked with finding the shortest path from the root node to a randomly selected leaf node, as in the forward prompt above. **b** Reverse condition: models were tasked with finding the shortest path from a randomly selected leaf node back to the root node.

C Automated LLM Annotation

We developed an automated pipeline for labeling our clusters, which corroborates our manual annotations. Specifically, for each cluster we randomly sampled 50 tokens, included the 15 preceding and following tokens as context, and formatted these into 50 examples for GPT-4o. Within each example, we highlighted the critical token using the notation `**[critical token]**`. Examples were ordered by response and then by token index within each response.

We include four examples below from Phi4-Reasoning's Cluster 41, which illustrate a heterogeneous set of tokens that are all used during the evaluation and comparison of reasoning steps.

Example 1: `0->5->4->1->3->2->0 is**[ best]**.`

`Perhaps:  0->5->1->3->4->2`

Example 2: `=121, 121+30=151, so that's 151**[,]** not better.`

`What if 0->2->5->1->4`

Example 3: `90. Remaining: F, then F->A: 50. But**[ wait]**, we still have F: from D, F: D->F =`

Example 4: `+12+11+45+17+21=120, we already**[ computed]** that.`

`Wait, check: 0->4->2->5->`

We employed three prompting methods to extract labels for each cluster: Prompt 1 emphasized the contextual usage of critical tokens, while Prompt 2 included each 31-token excerpt without highlighting the critical tokens. Prompt 3 served as an alternative to Prompt 1, placing greater emphasis on the critical tokens themselves, rather than their surrounding context. We found that Prompts 1 and 2 yielded the most accurate labels, aligning closely with our manual annotations. The full prompts are provided below:

Prompt 1 - Token-Centric: This text includes excerpts from a language model solving various traveling salesperson problems. It includes tokens highlighted with double asterisks and brackets (e.g., **[token]**), with the surrounding tokens provided as context. Consider the context in which each highlighted token appears, and provide a single phrase (10 words or fewer) that describes their algorithmic role or roles across the whole set. The descriptive label should be specific, because it will later be contrasted with labels for tokens from other excerpts. The label may relate to a specific type of operation, reasoning strategy, heuristic, organizational principle, class of words, numbers relevant to the problem, tokens that appear in a specific context, etc. Remember that the label should be based on the context of the highlighted tokens.

Prompt 2- Context-Centric: This text includes excerpts from a language model solving various traveling salesperson problems. Consider all of the provided examples, and provide a single phrase (10 words or fewer) that describes the algorithmic role or roles common across the examples. The descriptive label should be specific, because it will later be contrasted with labels for other excerpts. The label may relate to a specific type of operation, reasoning strategy, heuristic, organizational principle, class of words, numbers relevant to the problem, tokens that appear in a specific context, etc.

Prompt 3 - Token-Centric (Alternative to Prompt 1): This text includes excerpts from a language model solving various traveling salesperson problems. It includes various critical tokens highlighted with double asterisks and brackets (e.g., **[critical token]**), with the surrounding tokens provided as context. Consider each highlighted token in the text, and provide a single phrase (10 words or fewer) that describes the entire set of critical tokens or encapsulates their algorithmic role or roles. The descriptive label should be specific, because it will later be contrasted with labels for tokens from other excerpts. The label may relate to a specific type of operation, reasoning strategy, heuristic, organizational principle, class of words, numbers relevant to the problem, tokens that appear in a specific context, etc. Remember that the label should be based on the highlighted tokens.

C.1 Labels of all clusters for Phi-4-Reasoning responses on TSP

We used our automated pipeline to assign labels to all cluster on the clustering model fit to Phi-4-Reasoning responses on TSP (see Table 3), sorting these labels by their associated expressivity. We excluded all clusters that were expressed in fewer than two tokens on average, resulting in a total of 46 clusters. Our results highlight that our pipeline demonstrates a strong agreement with our manual labels and, furthermore, that our selected clusters generally have high associated expressivity. To better understand how many clusters correspond to distinct algorithmic primitives, we used an LLM-based pipeline to further sort them into specific groups. This revealed eight clusters that were better characterized by linguistic patterns than algorithmically distinct roles:

- **Algorithmic Clusters:**
  - High-Level Strategies: 18, 37, 41, 35, 26, 6
  - Arithmetic Operations: 15, 11, 34, 42, 4, 13, 16, 8
  - Distance/Cost Calculation: 9, 0, 17, 19, 22, 23
  - Path Construction/Evaluation: 3, 14, 1, 38, 25
  - Edge/Path Operations: 5, 27, 33, 45, 47
  - Route Optimization: 31, 39, 46, 44, 21, 10
  - Organizational/Workflow: 30, 43
- **Linguistic Clusters:**
  - Notation/Punctuation: 12, 29, 24
  - Identifiers/Labels: 7, 40, 20

- Structural Markers: 32, 48

Table 3: Annotated labels for the clustering model fit to TSP, showing both manual labels and the two distinct automated labels. Clusters are ranked according their expressivity (number of unique tokens expressing the cluster, divided by the radius of the cluster in the latent space)

Rank	Cluster	Manual Label	Automated label (Token-centric)	Automated label (Context-centric)	Expressivity
1	18		Exploration of alternative routes and possibilities	Evaluating candidate routes and calculating total distances.	2.58
2	30	produce_final_answer	Output formatting with `<final_answer>` tag	Output formatting with `<final_answer>` tag.	1.17
3	41	compare_or_verify_2	Route exploration and modification suggestions	Route evaluation and improvement through swaps and recalculations.	1.11
4	37	nearest_neighbor	”Selecting best path based on edge weights”	”Selecting best path based on edge costs.”	0.94
5	19		Path and distance notation	Calculating and comparing path distances.	0.82
6	7		Key terms in TSP problem description	Shortest path computation in Traveling Salesman Problem (TSP).	0.65
7	9		City and path identifiers	Distance calculations between city pairs.	0.56
8	35	compare_or_verify	Proposed alternative routes	Route optimization and evaluation in TSP solutions.	0.35
9	15	compute_distance	Summation and accumulation of path costs	Addition and summation of route distances.	0.32
10	40		City and path descriptors	City-to-city distance representation.	0.32
11	11		Intermediate cumulative sum calculations	Incremental distance calculations with comparisons to find optimal path.	0.30
12	25		Delimiters and separators in route descriptions	Evaluating and selecting paths based on distance or cost.	0.30
13	0		Distance values between cities	City-to-city distance calculations	0.29
14	6		Exploring alternative routes and strategies for optimization.	Route exploration and optimization through swaps and recalculations.	0.28
15	29		Punctuation or spacing in route calculations	Calculating and updating total distances in routes.	0.27
16	17		Distances between city pairs	Pairwise city distances in traveling salesperson problems.	0.25

Continued on next page